CS450

Structure of Higher Level Languages

Lecture 2: Branching and function definitions

Tiago Cogumbreiro

Academic dishonesty

Plagiarism in University

Copying code from others is wrong because:

- you do not learn
- you risk being expelled
- you are risking the other person being expelled
- you risk not completing your degree
- you risk being put on a list of cheaters (other universities may reject your application)

UMass Boston

Plagiarism in the Industry

Is wrong, because:

- it is illegal
- you risk being dismissed from employment
- you risk being sued

UMass Boston

Copying code (when it is right)

- software licenses define clear rules on
  how you can copy, use, and change other people's code
- open source promotes sharing of code
  - attribution is important (unless public domain)
  - good way to land a job

UMass Boston

Plagiarism in CS 450

- student's responsibility to learn the Student's code of conduct
- we use plagiarism detection (renaming functions is not enough)
- we compare against solutions from past years (and instructor)
- be careful when working with others, any code sharing may trigger it
- the plagiarism detection tool can detect code sharing among students

UMass Boston

Evaluating a function call

Evaluation works from left-to-right from top-to-bottom

```
#racket lang
; Version 1:
(* 3.14159 (* 10 10))
; Version 2:
(* 3.14159 100)
;          ^^^- Evaluated (* 10 10)
; Version 3:
314.159
;^^^^^^- Evaluated (* 3.14159 100)
```

Evaluating a function call

Evaluating a function call

Evaluation works from left-to-right from top-to-bottom

```
#racket lang
; Version 1:
(* 3.14159 (* 10 10))
; Version 2:
(* 3.14159 100)
;          ^^^- Evaluated (* 10 10)
; Version 3:
314.159
;^^^^^^- Evaluated (* 3.14159 100)
```

Arithmetic expressions example

$$\Big((11 \cdot 15) + (14 + 4)\Big) + \Big(\frac{3}{9} - (14 \cdot 3)\Big)$$

Arithmetic expressions example

$$((11 \cdot 15) + (14 + 4)) + \left(\frac{3}{9} - (14 \cdot 3)\right)$$

```
(+
  (+
    (* 11 15)
    (+ 14 4))
  (-
    (/ 3  9)
    (* 14 3)))
```

A longer example

```
(+
  (+
    (* 11 15)
    (+ 14 4))
  (-
    (/ 3  9)
    (* 14 3)))
```

```
(+
  (+
    165
    (+ 14 4))
  (-
    (/ 3  9)
    (* 14 3)))
```

```
(+
  (+
    165
    18)
  (-
    (/ 3  9)
    (* 14 3)))
```

```
(+
  183
  (-
    (/ 3  9)
    (* 14 3)))
```

```
(+
  183
  (-
    1/3
    (* 14 3)))
```

```
(+
  183
  (-
    1/3
    42))
```

```
(+
  183
  -125/3)
```

```
424/3
```

Interpreting an error in Racket

What would happen if we call a function using the infix notation?

```
(3 / 9)
```

```
; application: not a procedure;
;  expected a procedure that can be applied to arguments
;   given: 3
; [,bt for context]
```

Line 1

The *subject* is `application`. Application is short for function application, aka **calling a function**.

The *symptom* is `not a procedure`. Something that should be a procedure is not. Recall, procedure = **function**.

Line 2

Calling a function requires a function, but we provided something else.

Line 3

We see what was given instead (number 3, rather than a function).

UMass Boston

Is this example a legal Racket program?

```
#lang racket
sin
```

Yes! `sin` is a variable, so a valid expression. Hence, Racket just prints what is in variable `sin`.

```
$ racket sin.rkt
#<procedure:sin>
```

> **Note:** In Racket lingo the word *procedure* is a synonym for function.

UMass Boston

Racket specification

```
program = #lang racket expression*
expression = value | variable | function-call | ...
value = number | ...
function-call = ( expression+ )
```

Logic

Values

- Numbers
- Void
- **Booleans**
- Lists
- ...

UMass Boston

Boolean, numeric comparisons

```
value = number | boolean | ...
boolean = #t | #f
```

- False: `#f`
- True: anything that is not `#f`
- Logical negation: function `(not e)` negates the boolean result of expression `e`
- Numeric comparisons: `<`, `>`, `≤`, `≥`, `=`

> To avoid subtle bugs, avoid using non-`#t` and non-`#f` values as true. In particular, **contrary to C** the number `0` corresponds to true. *Tip:* There is no numeric inequality operator. Instead, use `(not (= x y))`

UMass Boston

Logical and/or

```
expression = value | variable | function-call | or | and | ...
or = ( or expression* )
and = ( and expression* )
```

- Logical-and with short-circuit: `and` (0 or more arguments, 0-arguments yield `#t`)
- Logical-or with short-circuit: `or` (0 or more arguments, 0-arguments yield `#f`)

UMass Boston

Boolean examples

Operations `and`/`or` *accept multiple parameters.* Rectangle intersection:

```
(and (< a-left b-right)
     (> a-right b-left)
     (> a-top b-bottom)
     (< a-bottom b-top))
```

As an example of **short-circuit** logic, the expression

```
(or #t (f x y z))
```

evaluates to `#t` and does **not** evaluate `(f x y z)`. Recall that `and` also short-circuits.

Branching

Branching with `cond`

`cond` evaluates each branch sequentially until the *first* branch's condition evaluates to true.

```
expression = value | variable | function-call | or | and | cond
cond = ( cond branch+ )
branch = [ condition expression ]
condition = expression | else
```

Example

If `x` is greater than 3 returns `100`, otherwise if `x` is between 1 and 3 returns `200`, otherwise returns `300`:

```
(cond [(> x 3) 100]
      [(> x 1) 200]
      [else 300])
```

Creating variables

Variable definition

A definition *binds* a variable to the result of evaluating an expression down to a value.

```
( define variable expression )
```

Examples

```
#lang racket
(define pi 3.14159)
pi
(* pi 2)
```

```
$ racket def-val.rkt
3.14159
6.28318
```

UMass Boston

Revisiting the language specification

A *program* consists of zero or more terms.

```
#lang racket
term*
```

A *term* is either an *expression* or a *definition*.

```
term = expression | definition
```

UMass Boston

If everything evaluates down to a value,

then what does `define` evaluate to?

Void

Definitions evaluate to `#<void>`, which is the only value that is not printed to the screen.

```
(define pi 3.14159)      ⟵ A definition evaluates to ⟶ #<void>
```

> The void value cannot be created directly. Another way of getting a void value `#<void>` is by calling function `(void)`.

Try running this program and confirm that its output is empty:

```
#lang racket
(void)
```

Evaluating variable definition

When we execute a Racket program, we have an *environment* to bookkeep each variable, that is a map from variable names to values.

```
(define pi 3.14159)
(* pi 2)
```

```
; pi = 3.14159
#<void>
;^^^^^^- Eval define
(* pi 2)
```

```
; pi = 3.14159
; Prints #<void>
(* pi 2)
```

```
; pi = 3.14159
(* 3.14159 2)
;  ^^^^^^^- Subst pi
```

```
; pi = 3.14159
6.28318
;^^^^^^- Eval func
```

```
; pi = 3.14159
; Print 6.28318
```

UMass Boston

Do variables evaluate?

> Variables are considered expressions, so the runtime must look up the value *bound* to a variable as one step of the evaluation.

UMass Boston

Beware of re-definitions

The following is legal Racket code:

```
#lang racket
(define pi 3.14159)
(* pi 2)
(define + #f)
(+ pi 2)
```

Redefinitions lead to subtle errors!

- Redefinitions produce subtle side-effects and may void existing assumptions
- As we will see, redefinitions also complicate the semantics and code analysis

UMass Boston

Exercises and recap

Exercise: conditionals

- The modulo operator (`%`) in Racket is function `modulo`
- The equality operator in Racket is `equal?`

Translate the following code to Racket:

```
n = 16
if n % 15 == 0:
    return "fizzbuzz"
if n % 3 == 0:
    return "fizz"
if n % 5 == 0:
    return "buzz"
return n
```

UMass Boston

Exercise: conditionals (solution)

```
(define n 16)
(cond [(equal? (modulo n 15) 0) "fizzbuzz"]
      [(equal? (modulo n 3) 0) "fizz"]
      [(equal? (modulo n 5) 0) "buzz"]
      [else n])
```

UMass Boston

Exercise: evaluation

```
#| How many evaluation steps should we expect? |#
(define x (* 10 2))
(+ x (* 4 2))
```

Solution

5 steps

```
; Step 1: eval *
(define x 20)
(+ x (* 4 2))
```

```
; Step 2: eval define
; x = 20
(+ x (* 4 2))
```

```
; Step 3: eval x
; x = 20
(+ 20 (* 4 2))
```

```
; Step 4: eval *
(+ 20 8)
```

```
; Step 5: eval +
28
```

UMass Boston

Function declaration

Function declaration

A function declaration creates an anonymous function and consists of:

- **parameters**: zero or more parameters (identifiers, known as symbols)
- **body** which consists of one or more terms

When calling a function we replace each parameter defined in the lambda by the corresponding argument. If the number of arguments is not the expected one, then we get an error. The return value of the function corresponds to the evaluation of the *last* term in the body (known as the **tail position**).

```
function-dec = ( lambda ( variable* ) term+)
```

We can define `circumference` as a function and parameterize the radius:

```
#lang racket
(define circumference (lambda (radius) (* 2 3.14159 radius)))
(circumference 2)
```

```
$ racket func.rkt
12.56636
```

Evaluating a `lambda`

```
(define circ
 (lambda (radius) (* 2 3.14159 radius)))
(circ 2)
```

```
; circ = lambda ...
#<void>
;^^^^^- Eval define
(circ 2)
```

```
; circ = lambda ...
; Prints #<void>
(circ 2)
```

```
; circ = lambda ...
((lambda (radius) (* 2 3.14159 radius)) 2)
;^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^- Subst circ
```

```
; circ = lambda ...
(* 2 3.14159 2)
;^^^^^^^^^^^^^^- Applied func
```

```
; circ = lambda ...
12.56636
```

```
; circ = lambda ...
; Prints 12.56636
```

For more information on evaluation, read Section 1.1.5 of SICP.

UMass Boston

Function definition

Function definition

Racket introduces a shorthand notation for defining functions.

```
( define (variable+ ) term+ )
```

Example

```
#lang racket
(define (circumference radius)
  (* 2 3.14159 radius)))
(circumference 2)
```

A function definition expects one or more variables (symbols). The first variable is the function variable. The remaining variables are the arguments of the function declaration. The one-or-more terms are the body of the function declaration.

Which is a short-hand for:

```
( define variable (lambda ( variable* ) term+ ))
```

Exercise

The McCarthy 91 function was invented by computer scientist John McCarthy to motivate formal verification.

$$M(n) = n - 10 \text{ if } n > 100$$
$$M(n) = M(M(n + 11)) \text{ if } n \leq 100$$

- Implement the function in Racket
- What is $M(99)$?

The McCarthy 91 function is equivalent to

$$M(n) = n - 10 \quad \text{if } n > 100$$
$$M(n) = 91 \quad \text{if } n \leq 100$$

UMass Boston

Homework example

Template excerpt

The homework template is in our Directory page:

```
(define ex1 'todo)
(define ex2 'todo)
(define ex3 'todo)

;; ...
```

Autograder Results

Results | Code

Sanity check (0.0/1.0)

```
Are you using the homework template?

I could not find the following definitions:
 * define-basic?
 * define-func?
 * define?
 * apply-args
 * apply-func
 * apply?
 * lambda-body
 * lambda-params
 * lambda?
 * bst-insert
 * tree-set-value
 * tree-set-right
 * tree-set-left
 * tree-value
 * tree-right
 * tree-left
 * tree-leaf
 * tree
 * ex3
 * ex2
 * ex1

Tip #1: try assigning a dummy value to each definition. For instance:
(define define-basic? #f)

Tip #2: ensure your definitions are made public. The first two lines of your file should be:
#lang racket
(provide (all-defined-out))
```

STUDENT

[redacted]

AUTOGRADER SCORE

0.0 / 24.0

FAILED TESTS

Sanity check (0.0/1.0)

Autograder Results

Results | Code

Exercise 1.a (0.0/1.5)

Exercise 1.b (0.0/3.5)

Exercise 2 (0.0/2.0)

Exercise 3. bst-insert (0.0/3.0)

Exercise 3. tree (0.0/0.5)

Exercise 3. tree-leaf (0.0/0.5)

Exercise 3. tree-left (0.0/0.5)

Exercise 3. tree-right (0.0/0.5)

Exercise 3. tree-set-left (0.0/0.5)

Exercise 3. tree-set-right (0.0/0.5)

Exercise 3. tree-set-value (0.0/0.5)

STUDENT

Tiago Cogumbreiro

AUTOGRADER SCORE

0.0 / 24.0

FAILED TESTS

Exercise 1.a (0.0/1.5)
Exercise 1.b (0.0/3.5)
Exercise 2 (0.0/2.0)
Exercise 3. bst-insert (0.0/3.0)
Exercise 3. tree (0.0/0.5)
Exercise 3. tree-leaf (0.0/0.5)
Exercise 3. tree-left (0.0/0.5)
Exercise 3. tree-right (0.0/0.5)
Exercise 3. tree-set-left (0.0/0.5)
Exercise 3. tree-set-right (0.0/0.5)
Exercise 3. tree-set-value (0.0/0.5)
Exercise 3. tree-value (0.0/0.5)
Exercise 4.a. lambda? (0.0/3.0)
Exercise 4.b. lambda-params (0.0/0.5)
Exercise 4.c. lambda-body (0.0/0.5)
Exercise 4.d. apply? (0.0/1.0)
Exercise 4.e. apply-func, 4.f. apply-args (0.0/1.0)
Exercise 4.g. define? (0.0/0.2)
Exercise 4.h. define-basic? (0.0/1.0)
Exercise 4.i. define-func? (0.0/2.8)

Rerun Autograder | Debug via SSH | Submission History | Download Submission | **Resubmit**